

PRODUCT REQUIREMENTS DOCUMENT

Multilingual Graph RAG Platform

Enterprise Knowledge Intelligence System

Version 1.0 · Personal Portfolio Project · 2025–2026
Target Market: DACH Region Enterprise · Germany-First Design

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Problem Statement	5
2.1 The Core Problem	5
2.2 Why Existing Solutions Fail	5
2.3 The Gap This Project Fills	5
3. Project Significance & Business Overview	6
3.1 Market Context — Why Germany, Why Now	6
3.2 Business Use Cases	6
3.3 Impact & Significance	6
4. Technical Architecture.....	8
4.1 System Overview	8
4.2 Layer-by-Layer Breakdown.....	8
Layer 1 — React Frontend	8
Layer 2 — FastAPI Backend.....	8
Layer 3 — Storage.....	9
Layer 4 — Infrastructure	9
5. Complete Folder Structure	10
5.1 Root Level	10
5.2 Backend — Complete.....	10
5.3 Frontend — Complete	11
5.4 Data Scripts, Infra & Docs	11
6. Product Requirements	13
6.1 Functional Requirements.....	13
FR-1: Multilingual Query	13
FR-2: Graph-Enhanced Retrieval.....	13
FR-3: Document Ingestion	14
FR-4: Answer Generation & Citations	14
FR-5: Authentication & Admin.....	14
6.2 Non-Functional Requirements	15
7. Data Sources	16
8. Build Roadmap	17
8.1 Technology Stack Reference.....	17
9. Neo4j Graph Schema.....	20
9.1 Node Labels	20

- 9.2 Relationship Types 20
- 10. API Contracts..... 22
- 11. Environment Variables (.env.example)..... 23
- 12. Getting Started — Day 1 Commands 24
- 13. Portfolio Positioning for Germany..... 25
 - 13.1 Skills This Project Demonstrates 25
 - 13.2 Target Companies & Roles..... 25
 - 13.3 How to Present This in Interviews..... 26

1. Executive Summary

A production-grade, open-source platform that enables enterprises to search, query, and reason over multilingual document corpora using vector-based retrieval and a knowledge graph — deployed as a full-stack web application with a React frontend.

The Multilingual Graph RAG Platform is a portfolio-grade, production-ready AI system that combines three of the most in-demand capabilities in the enterprise AI market today: multilingual retrieval-augmented generation (RAG), knowledge graph reasoning, and agentic query orchestration. Built on a modern Python/FastAPI backend, a React/TypeScript frontend, and a dual-database storage layer (Milvus + Neo4j), the platform solves the real-world problem of querying large, cross-language document repositories in a way that surfaces not just relevant text chunks, but also the structured relationships between entities across those documents.

This project is designed specifically to serve as a flagship portfolio piece for Data & AI Engineering roles in Germany, where the combination of EU regulatory compliance, multilingual enterprise environments, and graph-based knowledge management represents a massive, underserved market. The platform is built to be deployed to cloud infrastructure, containerised with Docker, and observable via Prometheus/Grafana — meaning it demonstrates end-to-end production engineering, not just model prototyping.

Attribute	Value
Project Name	Multilingual Graph RAG Platform
Version	1.0 (MVP)
Project Type	Personal Portfolio — Production-grade
Primary Stack	FastAPI · React · Milvus · Neo4j · PostgreSQL
Embedding Model	multilingual-e5-large (mE5) / LaBSE
Graph Database	Neo4j 5.x (Aura free tier for deploy)
Target Languages	German (DE), English (EN), French (FR), Polish (PL)
Primary Dataset	MultiEURLEX — EU legislation in 23 languages
Deployment Target	Docker Compose (local) · GCP Cloud Run (prod)
Estimated Build Time	14–18 weeks solo

2. Problem Statement

2.1 The Core Problem

Large organisations — governments, law firms, pharmaceutical companies, financial institutions, and manufacturing enterprises — maintain tens of thousands of documents across multiple languages. These documents are deeply interconnected: a regulation references an article, which cites a decision, which amends an earlier directive. Today, knowledge workers must manually cross-reference these documents using keyword search tools that are language-specific, semantically shallow, and relationship-blind.

The result is staggering inefficiency. A compliance officer at a German bank must search GDPR text in German, cross-reference relevant ECJ decisions in French, and link those findings to their internal English-language policies — manually, using three separate systems, with no guarantee of completeness. A single compliance review that could be completed in minutes with intelligent retrieval takes days.

2.2 Why Existing Solutions Fail

Existing Approach	Limitation
Keyword search (Elasticsearch)	Language-specific, no semantic understanding, no relationship traversal
Single-language RAG	Cannot query across DE/EN/FR documents simultaneously
Generic chatbots (ChatGPT)	No grounding in private documents, no citation, hallucination risk
Manual cross-referencing	Slow, error-prone, not scalable, requires expert knowledge
Siloed document systems	No awareness of entity relationships across documents

2.3 The Gap This Project Fills

No open-source, deployable, full-stack solution exists that combines all three of: (1) cross-lingual dense retrieval, (2) knowledge graph entity linking and multi-hop traversal, and (3) a modern web UI — in a single production-ready project. This platform fills that exact gap.

3. Project Significance & Business Overview

3.1 Market Context — Why Germany, Why Now

Germany is the largest economy in Europe and operates in one of the world's most complex regulatory environments. The DACH region (Germany, Austria, Switzerland) is home to thousands of enterprises subject to EU regulations — GDPR, AI Act, NIS2 Directive, ESG reporting — all of which are published in multiple official EU languages. The volume of regulatory text is growing at ~15% per year.

At the same time, German enterprise AI adoption is accelerating. According to Bitkom, over 68% of German companies with more than 500 employees are actively investing in AI systems in 2024–2025, with knowledge management and compliance automation being the top two use cases. The demand for Data & AI Engineers who can build production-grade RAG systems is outpacing supply by a significant margin.

3.2 Business Use Cases

Industry	Use Case	Business Value
Legal & Compliance	EU regulatory Q&A	Reduce compliance review time from days to minutes; surface cross-regulation conflicts automatically
Pharmaceutical	Clinical trial literature	Query drug interactions and clinical evidence across EN/DE/FR research papers with entity-aware graph traversal
Automotive / Manufacturing	Technical documentation	Engineers query standards, safety recalls, and part specifications across DE/EN/PL manuals in a single interface
Financial Services	Internal policy search	Unified search over internal policies, BaFin directives, and ECB publications with traceable citations
Public Sector / Government	Legislation navigation	Government staff navigate EUR-Lex and national law simultaneously, with automatic entity linking across jurisdictions

3.3 Impact & Significance

- Reduces expert research time on cross-lingual regulatory questions from hours to seconds
- Eliminates language barrier as a reason for missed compliance obligations
- Surfaces hidden entity relationships that keyword search cannot find — e.g., 'all regulations that mention company X and refer to Article 17 GDPR'
- Provides grounded, cited answers — eliminating the hallucination risk of general-purpose LLMs

- Deployable: not a Jupyter notebook, but a real web app with auth, async ingestion, and monitoring

4. Technical Architecture

4.1 System Overview

The platform is composed of four architectural layers: a React/TypeScript frontend, a FastAPI Python backend, a dual-database storage layer, and a containerised infrastructure layer. The frontend communicates with the backend over REST and WebSocket. The backend orchestrates a RAG pipeline that combines multilingual dense retrieval from Milvus with entity-aware graph traversal from Neo4j, before passing enriched context to an LLM for answer generation.

4.2 Layer-by-Layer Breakdown

Layer 1 — React Frontend

Component	Description
Search UI	Language-aware query bar with automatic language detection, model selector, and filter panel
Results Panel	Streamed answer display with source chunks, confidence scores, and citation links
Graph Explorer	Interactive force-directed graph (react-force-graph) for entity/relationship visualisation
Upload Zone	Drag-and-drop document uploader with real-time ingestion status via WebSocket
Admin Dashboard	Ingestion job queue, language distribution charts, system health metrics
State Management	Zustand stores for query state, graph state, and ingestion state
API Layer	Axios client with interceptors, retry logic, and JWT token refresh

Layer 2 — FastAPI Backend

Service	Description
API Gateway	JWT authentication, CORS, rate limiting, request routing to pipeline services
Language Detection	fastText lid.176.bin model — detects language before routing to correct embedding model
Retrieval Service	multilingual-e5-large embeddings → Milvus ANN search → top-k chunk retrieval
Reranking Service	Cross-encoder (ms-marco-MiniLM) reranks retrieved chunks by query relevance

Graph Query Service	Cypher multi-hop queries on Neo4j — surfaces entity relationships and document connections
RAG Orchestrator	Fuses vector results + graph context → builds prompt → calls LLM → streams response
Ingestion Pipeline	PDF/XML/HTML parsing → semantic chunking → NER extraction → embedding → graph building
Celery Workers	Async task queue for document ingestion, graph building, and re-indexing jobs
Monitoring	Prometheus metrics endpoint — latency, retrieval hit rate, queue depth, error rate

Layer 3 — Storage

Database	Type	Stores
Milvus 2.4	Vector DB	1024-dim mE5 embeddings, chunk metadata, language tags, document IDs
Neo4j 5.x	Graph DB	Entities (persons, orgs, laws, articles), relationships (CITES, AMENDS, REFERS_TO), document nodes
PostgreSQL 16	Relational DB	Users, documents, ingestion jobs, query history, audit logs, API keys
Redis 7	Cache / Queue	Celery task queue, query result cache, session tokens, rate limit counters

Layer 4 — Infrastructure

Tool	Purpose
Docker Compose	Local development — spins up all 6 services (API, frontend, Milvus, Neo4j, Postgres, Redis) with a single command
GitHub Actions	CI pipeline: lint, type-check, pytest, Docker build, push to registry
GCP Cloud Run	Serverless container deployment — zero idle cost for portfolio demo
Prometheus + Grafana	Metrics collection and dashboarding — latency, error rate, queue depth
Alembic	Database migration management for PostgreSQL schema versioning

5. Complete Folder Structure

The following is the complete file and folder structure for the project. Every file listed should be created. Files marked with (*) are stubs that will be expanded in later phases.

5.1 Root Level

```

multilingual-graph-rag/ |— README.md                      # Project overview, setup,
demo screenshots |— .env.example                          # Environment variable template
|— .env.local                      # Local dev secrets (gitignored) |— .gitignore
|— Makefile                        # Dev commands: make dev, make test, make build
|— docker-compose.yml              # Local dev stack (all services) |— docker-
compose.prod.yml                  # Production overrides |— backend/ |— frontend/ |— data/
|— infra/ |— docs/ |— .github/

```

5.2 Backend — Complete

```

backend/ |— Dockerfile |— requirements.txt                # Pinned dependencies |—
pyproject.toml                # Ruff, mypy, pytest config |— .env.example |— app/
| |— __init__.py | |— main.py                            # FastAPI app factory,
lifespan hooks | |— dependencies.py                       # Shared DI: db sessions, auth user
| | | |— api/ | | |— __init__.py | | |— vl/ | | |— __init__.py |
| | | |— router.py | | |— aggregates all route prefixes | | |—
routes/ | | | |— __init__.py | | | |— query.py            # POST
/query - main RAG endpoint | | | |— ingest.py             # POST /ingest - upload
+ start job | | | |— graph.py                             # GET /graph/entities, /graph/subgraph
| | | |— documents.py | | | |— CRUD for ingested documents | | | |—
jobs.py | | | |— GET ingestion job status | | | |— health.py | | | |—
GET /health, /metrics | | | |— auth.py                   # POST /login, /refresh,
/logout | | | |— schemas/ | | | |— __init__.py | | | |—
query.py | | | |— QueryRequest, QueryResponse, ChunkResult | | | |—
ingest.py | | | |— IngestRequest, JobStatus | | | |— graph.py | | | |—
EntityNode, RelationshipEdge, Subgraph | | | |— document.py | | | |—
DocumentRecord, DocumentCreate | | | |— auth.py          # LoginRequest,
TokenResponse | | | |— core/ | | | |— __init__.py | | | |— config.py
# Pydantic Settings - all env vars | | | |— security.py   # JWT
encode/decode, password hashing | | | |— logging.py       # Structured JSON
logging (structlog) | | | |— exceptions.py                # Custom HTTP exception classes
| | | |— middleware.py | | | |— CORS, rate limiting, request ID | | | |—
pipeline/ | | | |— __init__.py | | | |— lang_detect.py     # Language
detection (fastText) | | | |— embedder.py                 # mE5/LaBSE embedding wrapper
| | | |— retriever.py | | | |— Milvus ANN retrieval service | | | |—
reranker.py | | | |— Cross-encoder reranking (ms-marco) | | | |—
graph_enricher.py | | | |— Fetch graph context for retrieved chunks | | | |—
generator.py | | | |— LLM call + prompt builder | | | |— rag_chain.py
# Main RAG orchestration: detect-retrieve-rerank-graph-generate | | | |— streaming.py
# SSE streaming response helper | | | |— graph/ | | | |— __init__.py | | | |—
neo4j_client.py | | | |— Neo4j driver singleton + connection pool | | | |—
cypher_queries.py | | | |— All Cypher query strings (parameterised) | | | |—
entity_linker.py | | | |— Link NER entities to existing graph nodes | | | |—
graph_builder.py | | | |— Build graph from NER output + doc metadata | | | |—
schema.py | | | |— Node labels, relationship types, constraints | | | |—
ingestion/ | | | |— __init__.py | | | |— parsers/ | | | |— __init__.py | | | |—
| | | |— base.py | | | |— Abstract parser interface | | | |— pdf_parser.py
# PyMuPDF - extract text + structure | | | |— xml_parser.py | | | |— lxml - EUR-
Lex Formex XML | | | |— html_parser.py | | | |— BeautifulSoup - web documents |
| | | |— chunker.py | | | |— Semantic chunking (sentence-transformers + spaCy) |
| | | |— ner_extractor.py | | | |— spaCy multilingual NER (xx_ent_wiki_sm) | | | |—
pipeline.py | | | |— Ingestion orchestration: parse-chunk-ner-embed-store | | | |—

```

```

└─ tasks.py # Celery task definitions | | | └─ db/ | | | └─
└─ init_.py | | | └─ postgres.py # SQLAlchemy async engine + session |
└─ milvus.py # pymilvus client + collection management | | | └─
redis.py # Redis client (aioredis) | | | └─ models/ | | | └─
└─ init_.py | | | └─ user.py # User ORM model | | | └─
document.py # Document ORM model | | | └─ job.py #
IngestionJob ORM model | | | └─ query_log.py # QueryLog ORM model | |
└─ migrations/ | | | └─ env.py # Alembic env | | | └─
versions/ # Migration scripts | | | └─ workers/ | | | └─ init_.py
└─ celery_app.py # Celery app factory + broker config | | | └─
monitor.py # Celery worker health + Prometheus metrics | | | └─ tests/ |
└─ init_.py | | | └─ conftest.py # Pytest fixtures (test DB, mock
embedder) | | | └─ unit/ | | | └─ test_lang_detect.py | | | └─ test_chunker.py | |
└─ test_ner_extractor.py | | | └─ test_embedder.py | | | └─ test_rag_chain.py | |
└─ integration/ | | | └─ test_query_endpoint.py | | | └─ test_ingest_endpoint.py | |
└─ test_graph_queries.py | | | └─ e2e/ | | | └─ test_full_pipeline.py

```

5.3 Frontend — Complete

```

frontend/ └─ Dockerfile └─ package.json └─ vite.config.ts └─ tsconfig.json └─
tailwind.config.ts └─ postcss.config.js └─ index.html └─ public/ | └─ favicon.ico
└─ src/ └─ main.tsx # React entry point, router, providers
└─ App.tsx # Root component + route definitions | | | └─
components/ | | | └─ layout/ | | | └─ Navbar.tsx | | | └─ Sidebar.tsx
| | | └─ PageWrapper.tsx | | | └─ search/ | | | └─ SearchBar.tsx #
Query input + lang selector + submit | | | └─ SearchBar.test.tsx | | | └─
LanguageSelect.tsx # DE/EN/FR/PL/Auto dropdown | | | └─ FilterPanel.tsx
# Date range, doc type, language filter | | | └─ results/ | | | └─
ResultsPanel.tsx # Main results container | | | └─ AnswerStream.tsx #
SSE streaming answer renderer | | | └─ ChunkCard.tsx # Source chunk with
citation | | | └─ LanguageBadge.tsx # DE/EN/FR flag + lang label | | | └─
└─ ScoreBadge.tsx # Relevance score display | | | └─ graph/ | | | └─
GraphViewer.tsx # react-force-graph-2d wrapper | | | └─ GraphControls.tsx
# Zoom, filter, depth controls | | | └─ EntityTooltip.tsx # Hover tooltip for
graph nodes | | | └─ RelationshipLegend.tsx | | | └─ upload/ | | | └─
UploadZone.tsx # react-dropzone + progress tracking | | | └─ JobStatus.tsx
# WebSocket job status display | | | └─ DocumentList.tsx # Uploaded
documents table | | | └─ ui/ | | | └─ Button.tsx | | | └─ Input.tsx
| | | └─ Badge.tsx | | | └─ Spinner.tsx | | | └─ Toast.tsx
# react-hot-toast wrapper | | | └─ ErrorBoundary.tsx | | | └─ pages/ | |
└─ SearchPage.tsx # Main search experience | | | └─ GraphExplorerPage.tsx
# Full-page graph visualisation | | | └─ AdminPage.tsx # Ingestion
management + metrics | | | └─ DocumentsPage.tsx # Document library browser
└─ LoginPage.tsx # JWT login form | | | └─ hooks/ | | | └─
useSearch.ts # Query → API → result state | | | └─ useGraph.ts
# Graph fetch + selection state | | | └─ useIngest.ts # File upload +
WebSocket job tracking | | | └─ useAuth.ts # JWT login/logout/refresh
└─ useStreamingAnswer.ts # SSE stream → text accumulator | | | └─ store/
└─ queryStore.ts # Zustand: query, results, loading | | | └─
graphStore.ts # Zustand: graph data, selected entity | | | └─
authStore.ts # Zustand: user, tokens | | | └─ uiStore.ts
# Zustand: sidebar, theme, toasts | | | └─ api/ | | | └─ client.ts
# Axios instance, interceptors, refresh | | | └─ search.ts #
searchDocuments(), streamQuery() | | | └─ graph.ts # getSubgraph(),
getEntity() | | | └─ ingest.ts # uploadDocument(), getJobStatus()
| | | └─ documents.ts # listDocuments(), deleteDocument() | | | └─
auth.ts # login(), logout(), refreshToken() | | | └─ types/
└─ query.types.ts | | | └─ graph.types.ts | | | └─ document.types.ts
auth.types.ts

```

5.4 Data Scripts, Infra & Docs

```

data/ | raw/ | | multieurlex/ # Downloaded MultiEURLEX parquet
files | | eurlex_xml/ # EUR-Lex Formex XML bulk download |
processed/ | | chunks/ # Processed chunks (JSONL) | |
embeddings/ # Pre-computed embeddings cache | scripts/ |
download_multieurlex.py # HuggingFace datasets download |
download_eurlex_xml.py # EUR-Lex OAI-PMH bulk download |
ingest_to_milvus.py # Batch embed + load into Milvus | build_graph.py
# Batch NER + Neo4j graph population | validate_data.py # Data
quality checks infra/ | docker-compose.yml # (symlink to root) |
nginx/ | | nginx.conf # Reverse proxy for prod | k8s/
# Optional Kubernetes manifests | | deployment.yaml | | service.yaml | |
configmap.yaml | monitoring/ | | prometheus.yaml # Scrape config
| grafana/ | | dashboards/ | | rag_platform.json # Pre-built
Grafana dashboard docs/ | architecture.md # ADRs and design
decisions | api_reference.md # OpenAPI endpoint docs |
data_pipeline.md # Ingestion pipeline walkthrough | graph_schema.md
# Neo4j node/relationship schema | deployment.md # Cloud deploy
instructions | demo_queries.md # 20 example queries with expected
results .github/ | workflows/ | | ci.yml # Lint + test on
PR | deploy.yml # Build + push + deploy on main merge

```

6. Product Requirements

6.1 Functional Requirements

FR-1: Multilingual Query

ID	Requirement	Priority	Acceptance Criteria
FR-1.1	Detect query language automatically	Must Have	fastText detects DE/EN/FR/PL with >95% accuracy on test set
FR-1.2	Retrieve relevant chunks across all languages from a single query	Must Have	English query returns relevant DE, FR, PL chunks within top-10 results
FR-1.3	Display language badge on every result chunk	Must Have	Every ChunkCard shows correct language flag and ISO code
FR-1.4	Allow manual language override	Should Have	Language selector in SearchBar forces retrieval from specified language
FR-1.5	Support cross-lingual question answering	Must Have	Answer is generated in query language regardless of source document languages

FR-2: Graph-Enhanced Retrieval

ID	Requirement	Priority	Acceptance Criteria
FR-2.1	Extract named entities from ingested documents	Must Have	spaCy NER extracts ORG, PER, LAW, LOC entities; >80% F1 on EUR-Lex test set
FR-2.2	Build knowledge graph from extracted entities	Must Have	Neo4j graph contains document nodes, entity nodes, and typed relationships
FR-2.3	Perform multi-hop graph traversal on query	Must Have	Query for entity returns 2-hop neighbour context in <500ms
FR-2.4	Display entity relationship graph in UI	Must Have	GraphViewer renders force-directed graph with clickable nodes

FR-2.5	Fuse graph context with vector retrieval results	Must Have	RAG chain combines top-k chunks + graph neighbourhood before generation
--------	--	-----------	---

FR-3: Document Ingestion

ID	Requirement	Priority	Acceptance Criteria
FR-3.1	Upload PDF, XML, HTML documents via UI	Must Have	UploadZone accepts all three formats; rejects unsupported types with error
FR-3.2	Show real-time ingestion status	Must Have	WebSocket pushes job progress (parsing/chunking/embedding/graphing) to UI
FR-3.3	Chunk documents semantically	Must Have	Chunks respect sentence boundaries; average chunk size 256–512 tokens
FR-3.4	Async background ingestion	Must Have	Upload returns immediately; Celery worker processes in background
FR-3.5	List and delete ingested documents	Should Have	DocumentsPage shows all docs with size, language, date, delete action

FR-4: Answer Generation & Citations

ID	Requirement	Priority	Acceptance Criteria
FR-4.1	Stream LLM-generated answer to UI	Must Have	SSE stream delivers tokens in <100ms first-token latency on local model
FR-4.2	Cite source chunks in answer	Must Have	Every factual claim links to a source chunk with document name and page
FR-4.3	Display relevance score per chunk	Should Have	ScoreBadge shows normalised 0–1 retrieval score on each ChunkCard
FR-4.4	Support configurable retrieval depth	Should Have	User can set top-k (5/10/20) and reranking toggle in FilterPanel

FR-5: Authentication & Admin

ID	Requirement	Priority	Acceptance Criteria
FR-5.1	JWT-based login / logout	Must Have	Unauthenticated requests to protected routes return 401
FR-5.2	Admin dashboard with ingestion metrics	Must Have	AdminPage shows job queue length, doc count by language, error rate
FR-5.3	Query history per user	Should Have	Last 50 queries retrievable from PostgreSQL query_log table
FR-5.4	API key generation for programmatic access	Nice to Have	Admin can generate/revoke API keys in AdminPage

6.2 Non-Functional Requirements

Category	Requirement	Target
Performance	Query latency (retrieval + generation)	P95 < 3 seconds end-to-end for top-10 retrieval + 200-token answer
Performance	Ingestion throughput	100-page PDF processed in <60 seconds on a 4-core dev machine
Scalability	Document corpus size	Support up to 100,000 documents / 2M chunks in Milvus without schema changes
Reliability	API uptime	99.5% uptime target; health check endpoint returns 200 within 1s
Security	Authentication	All endpoints except /health require valid JWT; tokens expire in 24h
Observability	Metrics coverage	All pipeline stages emit Prometheus counters + histograms; Grafana dashboard pre-built
Portability	Local setup time	git clone + make dev starts all services in <5 minutes on a clean machine with Docker

7. Data Sources

The following datasets are free, publicly available, and directly usable without scraping or API rate limits. They represent real enterprise-quality content that makes the portfolio project credible.

Dataset	Languages	Size	Access & Notes
MultiEURLEX (PRIMARY)	23 EU languages	65,000 laws	pip install datasets; load_dataset('multi_eurlex') — HuggingFace, CC-BY 4.0
EUR-Lex XML Bulk	DE, EN, FR, PL	Full corpus	eur-lex.europa.eu/oj/direct-access.html — OAI-PMH API, free, Formex XML
OpenLegalData.de	German (DE)	~500K decisions	openlegaldata.io/api — REST API, open, German court decisions
Wikipedia Dumps	330 languages	Millions of articles	dumps.wikimedia.org — free, CC-BY-SA, good for graph entity seeding
arXiv bulk data	English (EN)	2M+ papers	info.arxiv.org/help/bulk_data — S3 access, CC-BY, good for technical demo

8. Build Roadmap

The project is broken into 5 phases, each producing a deployable increment. Total estimated effort for a solo developer: 14–18 weeks working part-time (~15–20 hours/week).

Phase	Weeks	Key Deliverables
Phase 1 Foundation	1–3	Docker Compose stack (all services), FastAPI skeleton, PostgreSQL models + Alembic, JWT auth (login/refresh), React app scaffold (Vite + Tailwind + Zustand), LoginPage, Navbar, CI pipeline (GitHub Actions), README with local setup instructions. Milestone: make dev starts all services; login works end-to-end.
Phase 2 Multilingual RAG	4–7	Language detection (fastText), mE5 embedding pipeline, Milvus collection setup + indexing, semantic chunker, MultiEURLEX data download + ingestion script, retrieval endpoint (POST /query), basic SearchPage + SearchBar + ResultsPanel with LanguageBadge + ChunkCard, SSE answer streaming. Milestone: cross-lingual query returns relevant chunks in DE/EN/FR from CLI and UI.
Phase 3 Graph Layer	8–11	spaCy multilingual NER, Neo4j graph builder (entity nodes + relationships), Cypher multi-hop query service, graph enricher (fuse with vector results), graph ingest tasks (Celery), GraphViewer component (react-force-graph-2d), GraphExplorerPage, entity tooltip + relationship legend. Milestone: query for 'GDPR Article 17' surfaces related directives, organisations, and court decisions in graph.
Phase 4 Ingestion UI & Admin	12–14	UploadZone with react-dropzone, WebSocket job status push, DocumentList + DocumentsPage, Celery ingestion pipeline (parse→chunk→embed→NER→graph), AdminPage with job queue metrics, query history (PostgreSQL), Prometheus metrics on all pipeline stages, Grafana dashboard JSON. Milestone: user can upload a PDF via UI and watch it appear in graph + search within 60 seconds.
Phase 5 Production & Polish	15–18	Reranker integration (ms-marco cross-encoder), FilterPanel (top-k, date range, language), ScoreBadge on chunks, GCP Cloud Run deploy (docker-compose.prod.yml), HTTPS via Cloud Run managed cert, Nginx reverse proxy, load testing (Locust), full E2E test suite, demo_queries.md (20 example queries), architecture docs, GitHub Pages demo, LinkedIn project post. Milestone: live public URL works; latency P95 < 3s.

8.1 Technology Stack Reference

Category	Library / Tool	Version / Notes
----------	----------------	-----------------

Backend framework	FastAPI	0.115.x — async, Pydantic v2
Embedding model	multilingual-e5-large	intfloat/multilingual-e5-large — 1024-dim, HuggingFace
Fallback embedding	LaBSE	sentence-transformers/LaBSE — 768-dim
Language detection	fastText	lid.176.bin — 176 languages, <1ms inference
NER	spaCy	xx_ent_wiki_sm (multilingual) + de_core_news_sm (German)
Vector database	Milvus	2.4.x — HNSW index, IVF_FLAT fallback
Graph database	Neo4j	5.x — Aura free tier for cloud deploy
Relational DB	PostgreSQL	16 — via SQLAlchemy 2.0 async
Cache / queue	Redis	7 — Celery broker + result backend
Task queue	Celery	5.x — async ingestion workers
Reranker	ms-marco-MiniLM-L-6-v2	cross-encoder, sentence-transformers
LLM (local)	Ollama + Mistral-7B	mistral:7b-instruct — free, runs on CPU/GPU
LLM (cloud)	OpenAI / Groq API	gpt-4o-mini or llama-3.1-70b — optional
Frontend	React 18 + TypeScript	Vite 5.x build tool
UI styling	Tailwind CSS	3.x — utility-first
State management	Zustand	4.x — lightweight
Graph visualisation	react-force-graph-2d	Canvas-based, 60fps force-directed graph
HTTP client	Axios	1.x — with interceptors
File upload	react-dropzone	14.x
Notifications	react-hot-toast	2.x
Testing (BE)	pytest + pytest-asyncio	7.x + 0.23.x
Testing (FE)	Vitest + React Testing Library	1.x + 14.x
Containerisation	Docker + Docker Compose	Docker 25.x
CI/CD	GitHub Actions	ubuntu-latest runners
Cloud deploy	GCP Cloud Run	Serverless containers, free tier available

Metrics	Prometheus + Grafana	prometheus_fastapi_instrumentator
DB migrations	Alembic	1.13.x
Linting (BE)	Ruff + mypy	0.6.x + 1.x
Linting (FE)	ESLint + Prettier	9.x + 3.x

9. Neo4j Graph Schema

9.1 Node Labels

Label	Key Properties	Description
:Document	id, title, language, source_url, celex_id, ingested_at	One node per ingested document; CELEX ID links to EUR-Lex
:Chunk	id, text, embedding_id, chunk_index, token_count, language	Granular text segment; embedding_id links to Milvus vector
:Entity	id, name, type, language, wikidata_id	Named entity (ORG, PER, LAW, LOC); type from spaCy NER
:LegalArticle	id, article_number, regulation_id, text	Specific article within a legal document
:Organisation	id, name, country, lei_code	Legal entity — company, government body, institution
:Person	id, name, nationality, role	Named individual extracted from documents
:Concept	id, name, definition, uri	Abstract concept node (e.g., GDPR, AI Act topic area)

9.2 Relationship Types

Relationship	Source → Target	Meaning
[:HAS_CHUNK]	Document → Chunk	Document contains this text chunk
[:MENTIONS]	Chunk → Entity	Chunk text mentions this entity
[:CITES]	Document → Document	Document explicitly cites another document
[:AMENDS]	Document → LegalArticle	Regulation amends a specific article
[:REFERS_TO]	Document → Concept	Document refers to this abstract concept
[:PART_OF]	LegalArticle → Document	Article belongs to this document
[:SUBJECT_OF]	Entity → Document	Entity is a primary subject of this document
[:RELATED_TO]	Entity → Entity	Two entities co-occur frequently or share context

[:TRANSLATED_FROM]	Document → Document	Document is a translation of another
---------------------	---------------------	--------------------------------------

10. API Contracts

All endpoints are prefixed with `/api/v1`. Authentication is JWT Bearer unless marked public.

Method	Path	Auth	Description & Body
POST	/auth/login	Public	Body: {email, password} → {access_token, refresh_token, expires_in}
POST	/auth/refresh	Public	Body: {refresh_token} → {access_token}
POST	/query	JWT	Body: {query, language?, top_k?, use_graph?} → SSE stream of QueryResponse
GET	/query/history	JWT	Returns last 50 queries for authenticated user
POST	/ingest	JWT	Multipart form: file + {title?, language?} → {job_id, status}
GET	/ingest/{job_id}	JWT	Returns job status: {job_id, stage, progress, error?}
GET	/documents	JWT	List all documents: [{id, title, language, chunk_count, ingested_at}]
DELETE	/documents/{id}	JWT	Delete document + chunks from Milvus + graph nodes
GET	/graph/entities	JWT	Query: ?name=&type=&lang= → [{id, name, type, degree}]
GET	/graph/subgraph/{entity_id}	JWT	Query: ?depth=2 → {nodes: [], edges: []} for GraphViewer
GET	/health	Public	Returns service health: {status, milvus, neo4j, postgres, redis}
GET	/metrics	Public	Prometheus text format metrics

11. Environment Variables (.env.example)

```
# — FastAPI — ENVIRONMENT=development
# development | production SECRET_KEY=your-jwt-secret-here # openssl rand -hex 32
ACCESS_TOKEN_EXPIRE_MINUTES=1440 ALLOWED_ORIGINS=http://localhost:5173 # — PostgreSQL
POSTGRES_HOST=localhost POSTGRES_PORT=5432
POSTGRES_DB=rag_platform POSTGRES_USER=raguser POSTGRES_PASSWORD=changeme # — Milvus
MILVUS_HOST=localhost MILVUS_PORT=19530
MILVUS_COLLECTION_NAME=multilingual_chunks # — Neo4j
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j NEO4J_PASSWORD=changeme NEO4J_DATABASE=neo4j # — Redis / Celery
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/1 CELERY_RESULT_BACKEND=redis://localhost:6379/2
# — Embedding Model —
EMBEDDING_MODEL=intfloat/multilingual-e5-large EMBEDDING_DIMENSION=1024
RERANKER_MODEL=cross-encoder/ms-marco-MiniLM-L-6-v2
FASTTEXT_MODEL_PATH=./models/lid.176.bin # — LLM
LLM_PROVIDER=ollama
# ollama | openai | groq OLLAMA_BASE_URL=http://localhost:11434 OLLAMA_MODEL=mistral:7b-
instruct OPENAI_API_KEY= # optional fallback GROQ_API_KEY=
# optional fallback # — Frontend (Vite) —
VITE_API_BASE_URL=http://localhost:8000/api/v1 VITE_WS_URL=ws://localhost:8000/ws
```

12. Getting Started — Day 1 Commands

Run these commands to go from zero to a running local development environment.

```
# 1. Clone and setup git clone https://github.com/YOUR_USERNAME/multilingual-graph-
rag.git cd multilingual-graph-rag cp .env.example .env.local # 2. Start all
infrastructure services docker compose up -d postgres milvus neo4j redis # 3. Install
backend dependencies cd backend python -m venv .venv && source .venv/bin/activate pip
install -r requirements.txt # 4. Run database migrations alembic upgrade head # 5.
Download embedding model (one-time, ~1.2GB) python -c "from sentence_transformers import
SentenceTransformer; SentenceTransformer('intfloat/multilingual-e5-large')" # 6.
Download fastText language model (one-time, ~900MB) wget -O models/lid.176.bin
https://dl.fbaipublicfiles.com/fasttext/supervised-models/lid.176.bin # 7. Start FastAPI
development server uvicorn app.main:app --reload --port 8000 # 8. In a new terminal -
start Celery worker celery -A app.workers.celery_app worker --loglevel=info # 9. In a
new terminal - start React frontend cd ../frontend npm install npm run dev # 10. Load
initial dataset (MultiEURLEX sample) cd ../data/scripts python download_multieurlex.py --
sample 1000 python ingest_to_milvus.py python build_graph.py # App is now running at: #
Frontend: http://localhost:5173 # API: http://localhost:8000 # API Docs:
http://localhost:8000/docs # Neo4j UI: http://localhost:7474 # Milvus UI:
http://localhost:3000 (attu)
```

Makefile shortcut: run 'make dev' from the project root to start all services and both servers in one command.

13. Portfolio Positioning for Germany

13.1 Skills This Project Demonstrates

Skill Area	What the Project Shows
LLM Engineering	Full RAG pipeline: embedding, retrieval, reranking, graph enrichment, generation, streaming
Data Engineering	Async ingestion pipeline, semantic chunking, NER, vector indexing, graph population
Backend Engineering	FastAPI async, Pydantic v2, SQLAlchemy 2.0, Celery, dependency injection, JWT
Frontend Engineering	React 18, TypeScript, Zustand, SSE streaming, WebSocket, force-directed graph
Database Engineering	Three DB types (vector, graph, relational) with appropriate data modelling
MLOps / DevOps	Docker Compose, GitHub Actions CI/CD, Prometheus metrics, Grafana dashboards
Cloud Deployment	GCP Cloud Run deploy from Docker; HTTPS, environment config, health checks
Multilingual NLP	Cross-lingual embeddings, language detection, multilingual NER — rare skill set
Graph Reasoning	Neo4j Cypher, entity linking, multi-hop traversal — high demand in Germany
System Design	Full architecture from ingestion to generation; async, observable, scalable

13.2 Target Companies & Roles

Company Type	Why This Project Resonates
Automotive (BMW, Bosch, Continental)	Technical documentation search across DE/EN/PL in graph-linked knowledge bases
Financial (Deutsche Bank, Commerzbank)	Regulatory compliance (BaFin, ECB, GDPR) cross-language search with citation
Consulting (McKinsey, Accenture Germany)	Enterprise knowledge management — exactly what they sell to clients
Legal Tech (Wolters Kluwer, LexisNexis DE)	Court decision + legislation graph traversal is their core product

SAP, Software AG, Celonis	Knowledge graph + AI — directly maps to their enterprise AI product roadmaps
AI Startups (Berlin/Munich)	Production-grade RAG stack is exactly what early-stage AI startups need to hire for

13.3 How to Present This in Interviews

1. Start with the problem: 'I built this because German enterprises operate in a multilingual regulatory environment and existing RAG solutions are language-specific and relationship-blind.'
2. Explain the two-layer retrieval: 'First, multilingual dense retrieval using mE5 embeddings finds semantically relevant chunks across DE/EN/FR documents. Then, Neo4j graph traversal surfaces related entities and documents that vector search alone would miss.'
3. Live demo the cross-lingual query: type a German question, show English source chunks appearing in results with correct citations.
4. Show the graph explorer: click an entity like 'GDPR Article 17', show its 2-hop neighbourhood connecting to related directives, organisations, and court decisions.
5. Walk through the architecture diagram: mention async ingestion, Celery workers, Prometheus metrics — this is the production engineering that separates this from a notebook project.
6. Close with deployment: show the live Cloud Run URL. A working URL on a CV is worth more than any description.

Key differentiator to state explicitly: 'Unlike most portfolio RAG projects that are single-language Jupyter notebooks, this is a deployed, multi-service web application with a graph reasoning layer, async ingestion, a React frontend, and production observability.'